PRACTICAL 1 - LINEAR (MIXED) MODELS

In this practical we are going to fit linear (mixed) models in `inlabru`. We are going to to:

- Fit a simple linear regression
- Fit a linear regression with discrete covariates and interactions
- Fit a linear mixed model.
- Compare models using DIC and WAIC

You can download the R-script of this practical by clicking the button below:

Start by loading useful libraries:

```
library(INLA)
library(patchwork)
library(inlabru)
library(tidyverse)
# load some libraries to generate nice plots
library(scico)
```

At the end of this exercise we are going to compare the different models we fit using DIC and WAIC. Therefore we need to tell the `bru()` function to compute those scores, we can do that by the `bru_options_set()` functions:

```
bru_options_set(control.compute = list(dic = T, waic = T))
```

This is a global option that will make `bru()` compute scores everytime. It is also possible to set the option locally as

```
fit = bru(cmp, lik,
          options = list(control.compute = list(dic = TRUE)))
```

1 Simple linear regression

We consider a simple linear regression model with Gaussian observations

$$y_i \sim \mathcal{N}(\mu_i, \sigma_y^2), \qquad i = 1, \dots, N$$

where σ_y^2 is the observation error, and the mean parameter μ_i is linked to the **linear predictor** (η_i) through an identity function:

$$\eta_i = \mu_i = \beta_0 + \beta_1 x_i.$$

Here $\mathbf{x} = (x_1, \dots, x_N)$ is a continuous covariate and β_0, β_1 are parameters to be estimated.

To finalize the Bayesian model we assign prior distribution as $\tau_y = 1/\sigma_y^2 \sim$ Gamma(a, b) and $\beta_0, \beta_1 \sim \mathcal{N}(0, 1/\tau_\beta)$ (we will use the default prior settings in `R-INLA` for now).

Question

What is the dimension of the hyperparameter vector and latent Gaussian field?
Answer
The hyperparameter vector has dimension 1, $\boldsymbol{\theta} = (\tau_y)$ while the latent Gaussian field $\boldsymbol{u} = (\beta_0, \beta_1)$ has dimension 2, 0 mean, and sparse precision matrix:

$$\boldsymbol{Q} = \begin{bmatrix} \tau_{\beta_0} & 0 \\ 0 & \tau_{\beta_1} \end{bmatrix}$$

Note that, since β_0 and β_1 are fixed effects, the precision parameters τ_{β_0} and τ_{β_1} are fixed.

Note

We can write the linear predictor vector $\boldsymbol{\eta} = (\eta_1, \dots, \eta_N)$ as

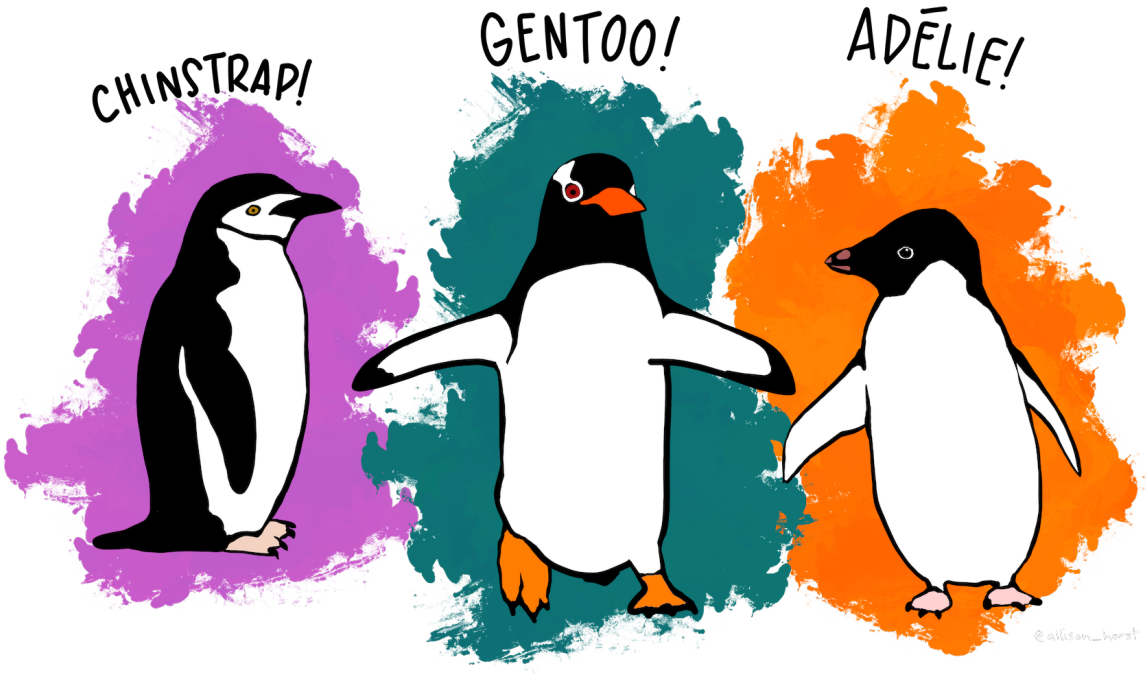
$$\boldsymbol{\eta} = \boldsymbol{A}\boldsymbol{u} = \boldsymbol{A}_1\boldsymbol{u}_1 + \boldsymbol{A}_2\boldsymbol{u}_2 = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \end{bmatrix} \beta_0 + \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \beta_1$$

Our linear predictor consists then of two components: an intercept and a slope.

2 Modelling penguins bodymass

Let's look at the dataset `penguins` in R.

These are data on adult penguins covering three species found on three islands in the Palmer Archipelago, Antarctica, including their size (flipper length, body mass, bill dimensions), and sex.



```
data("penguins")
glimpse(penguins)
```

```
Rows: 344
Columns: 8
$ species     <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, Ad~
$ island      <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torgersen, Tor~
$ bill_len    <dbl> 39.1, 39.5, 40.3, NA, 36.7, 39.3, 38.9, 39.2, 34.1, 42.0, ~
$ bill_dep    <dbl> 18.7, 17.4, 18.0, NA, 19.3, 20.6, 17.8, 19.6, 18.1, 20.2, ~
$ flipper_len <int> 181, 186, 195, NA, 193, 190, 181, 195, 193, 190, 186, 180,~
$ body_mass   <int> 3750, 3800, 3250, NA, 3450, 3650, 3625, 4675, 3475, 4250, ~
$ sex         <fct> male, female, female, NA, female, male, female, male, NA, ~
$ year        <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007~
```

The dataset contains the following variables:

- `species` a factor denoting penguin species (Adélie, Chinstrap and Gentoo)
- `island` a factor denoting island in Palmer Archipelago, Antarctica (Biscoe, Dream or Torgersen)
- `bill_length` a number denoting bill length (millimeters)
- `bill_depth_mm` a number denoting bill depth (millimeters)
- `flipper_length` an integer denoting flipper length (millimeters)
- `body_mass` an integer denoting body mass (grams)
- `sex` a factor denoting penguin sex (female, male)
- `year` and integer indicating the study year: (2007, 2008, 2009)

We are going to use the bodymass as our response variable. To make the interpretation easier we are going to express the bodymass in Kg instead of grams.

```
penguins$body_mass = penguins$body_mass/1000
```

In general, while INLA can fit models with missing observations in the response by computing their predictive distribution, it cannot directly handle missing observations in the covariates, since these enter the latent field. For this reason, we will remove rows with missing values before fitting.

```
penguins = penguins %>%  drop_na()
```

> **!** Important
>
> Note that you should be careful when removing NA's from your data. Dropping incomplete rows is generally safe when the values are missing at random. However, if missingness is related to the outcome or to the covariates themselves, discarding those rows can bias the results, so the pattern of missing data should always be inspected before removing anything.

2 Fitting a linear regression model with `inlabru`

In a first step we want to fit a simple linear regression model to link the body mass (`body_mass`) to flipper length (`flipper_len`).

$$y_i \sim \mathcal{N}(\mu_i, \sigma_y^2)$$
$$\mu_i = \beta_0 + \beta_i x_i$$

where y_i indicates the body mass and x_i the flipper length of penguin i.

Step1: Defining model components

The first step is to define the two model components: The intercept and the linear covariate effect.

Task

Define an object called `cmp` that includes and (i) intercept `beta_0` and (ii) and a linear effect `beta_1` of the `flipper_len`.
Take hint
The `cmp` object is here used to define model components. We can give them any useful names we like, in this case, `beta_0` and `beta_1`. You can remove the automatic intercept construction by adding a `-1` in the components
Click here to see the solution

```
cmp =  ~ -1 + beta_0(1) + beta_1(flipper_len, model = "linear")
```

Note

Note that the default Intercept term in the model can be excluded by typing `-1` in the model components. However, `inlabru` has automatic intercept that can be called by typing `Intercept()` , which is one of `inlabru` special names and it is used to define a global intercept, e.g.

```
cmp =  ~  Intercept(1) + beta_1(flipper_len, model = "linear")
```

Step 2: Build the observation model

The next step is to construct the observation model by defining the model likelihood. The most important inputs here are the `formula`, the `family` and the `data`.

Task

Define a linear predictor `eta` using the component labels you have defined on the previous task (i.e., `beta_0` and `beta_1`).
Take hint
The `eta` object defines how the components should be combined in order to define the model predictor.
Click here to see the solution

```
eta = body_mass ~  beta_0 + beta_1
```

The likelihood for the observational model is defined using the `bru_obs()` function.

Task

Define the observational model likelihood in an object called `lik` using the `bru_obs()` function.
Take hint
The `bru_obs` is expecting three arguments:

- The linear predictor `eta` we defined in the previous task
- The data likelihood (this can be specified by setting `family = "gaussian"`)
- The data set `penguins`

Click here to see the solution

```
lik = bru_obs(formula = eta,
              data = penguins,
              family = "gaussian")
```

Step 3: Fit the model

We fit the model using the `bru()` functions which takes as input the components and the observation model:

```
fit1 = bru(cmp, lik)
```

Step 4: Extract results

There are several ways to extract and examine the results of a fitted `inlabru` object.

The most natural place to start is to use the `summary()` function which gives access to some basic information about model fit and estimates

```
summary(fit1)
## inlabru version: 2.15.0
## INLA version: 26.08.22
## Latent components:
## beta_0: main = linear(1)
## beta_1: main = linear(flipper_len)
## Observation models:
##   Model tag: <No tag>
##     Family: 'gaussian'
##     Data class: 'data.frame'
##     Response class: 'numeric'
##     Predictor: body_mass ~ beta_0 + beta_1
##     Additive/Linear/Rowwise: TRUE/TRUE/TRUE
##     Used components: effect[beta_0, beta_1], latent[]
## Time used:
##     Pre = 0.841, Running = 1.45, Post = 0.127, Total = 2.41
## Fixed effects:
##            mean    sd 0.025quant 0.5quant 0.975quant   mode kld
## beta_0 -5.872 0.310     -6.480   -5.872     -5.263 -5.872   0
## beta_1  0.050 0.002      0.047    0.050      0.053  0.050   0
##
```

```
## Model hyperparameters:
##                                          mean    sd 0.025quant 0.5quant
## Precision for the Gaussian observations 6.50 0.504       5.55     6.49
##                                         0.975quant mode
## Precision for the Gaussian observations       7.53 6.46
##
## Deviance Information Criterion (DIC) ...............: 327.57
## Deviance Information Criterion (DIC, saturated) ....: 338.38
## Effective number of parameters .....................: 2.99
##
## Watanabe-Akaike information criterion (WAIC) ...: 327.53
## Effective number of parameters .................: 2.92
##
## Marginal log-Likelihood:  -187.71
##  is computed
## Posterior summaries for the linear predictor and the fitted values are computed
## (Posterior marginals needs also 'control.compute=list(return.marginals.predictor=TRUE)')
```

We can further inspect posterior summaries of hyperparameters, fixed and random effects by calling `model$summary.hyperpar`, `model$summary.fixed` and `model$summary.random` respectively.

Task

Produce posterior summaries of the fixed effects

Take hint

You can use the $ sign on a `bru` object and press the Tab key to see the different outputs stored in it.

Click here to see the solution

```
fit1$summary.fixed
```

```
              mean          sd  0.025quant    0.5quant  0.975quant        mode
beta_0 -5.87152801 0.310089584 -6.47995914 -5.87152970 -
5.26308728 -5.87152969
beta_1  0.05015047 0.001539261  0.04713022  0.05015048  0.05317067  0.05015048
                kld
beta_0 5.749734e-10
beta_1 5.748509e-10
```

2 Predictions with `inlabru`

Another way, which gives access to more complicated (and useful) output is to use the `predict()` function.

Below we take the fitted `bru` object and use the `predict()` function to produce predictions for η given a new set of values for the penguins flipper length

```
new_data = data.frame(flipper_len  = 170:240)
```

```
pred = predict(fit1, new_data, ~ beta_0 + beta_1,
               n.samples = 1000)
```

The `predict()` function generates samples from the fitted model and then summarizes those samples by computing some statistics like posterior mean, standard deviation, quantiles etc. In this case we set the number of samples to 1000.

> **Note**
>
> Note that we are predicting within the range of our covariate, as extrapolating beyond the observed data is risky because the model has no information there, so predictions rely entirely on the assumed functional form, and their uncertainty can be badly understated

We can plot the predictions for η together with the observed data:

```
pred %>% ggplot() +
  geom_line(aes(flipper_len,mean)) +
  geom_ribbon(aes(flipper_len, ymin = q0.025, ymax = q0.975), alpha = 0.5) +
  xlab("Flipper length") + ylab("body_mass") +
  geom_point(data = penguins, aes(flipper_len, body_mass))
```

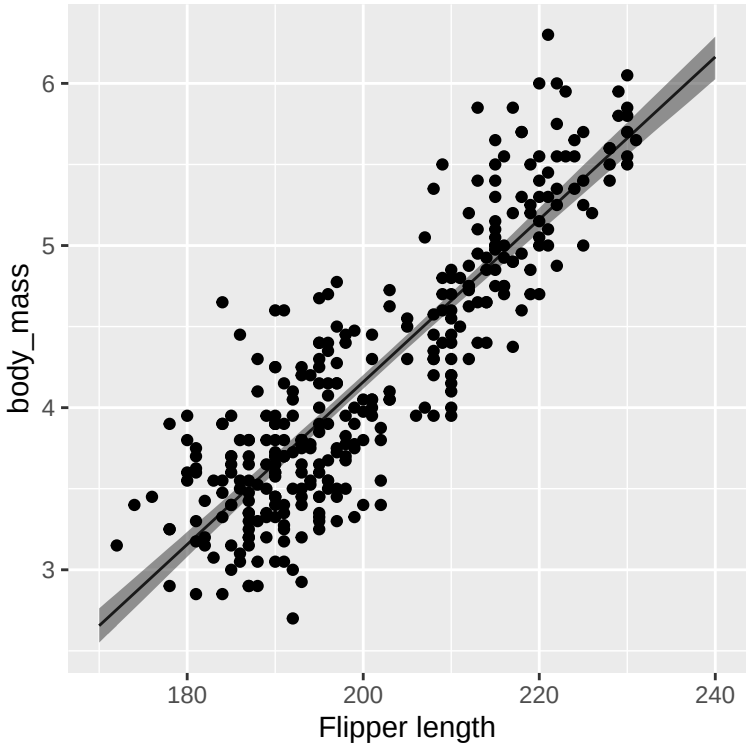


Figure 1: Data and 95% credible intervals

NOTE: The uncertainty we have computed now is relative to the prediction of η. If we want to predict new body mass, we need to add the observation (likelihood) uncertainty σ_y^2. We can do this using `predict()` again:

```
new_data = data.frame(flipper_len  = 170:240)
```

```
pred1 = predict(fit1, new_data,
                formula = ~ { eta = beta_0 + beta_1
                              sigma = sqrt(1/Precision_for_the_Gaussian_observations)
                              list(mean = eta,
                                   q1 = qnorm(0.025, mean = eta, sd = sigma),
                                   q2 =  qnorm(0.975, mean = eta, sd = sigma))
                            },
                n.samples = 1000)
```

Let's compare the two predictions:

```
ggplot() +
  geom_line(data = pred, aes(flipper_len,mean)) +
  geom_ribbon(data = pred,aes(flipper_len, ymin = q0.025, ymax = q0.975), alpha = 0.5) +
  geom_line(data = pred1$mean, aes(flipper_len,mean), color = "red") +
  geom_line(data = pred1$q1,aes(flipper_len, mean),
                color = "red") +
   geom_line(data = pred1$q2,aes(flipper_len, mean),
                color = "red") +
  xlab("flipper length") + ylab("body_mass") +
  geom_point(data = penguins, aes(flipper_len, body_mass))
```

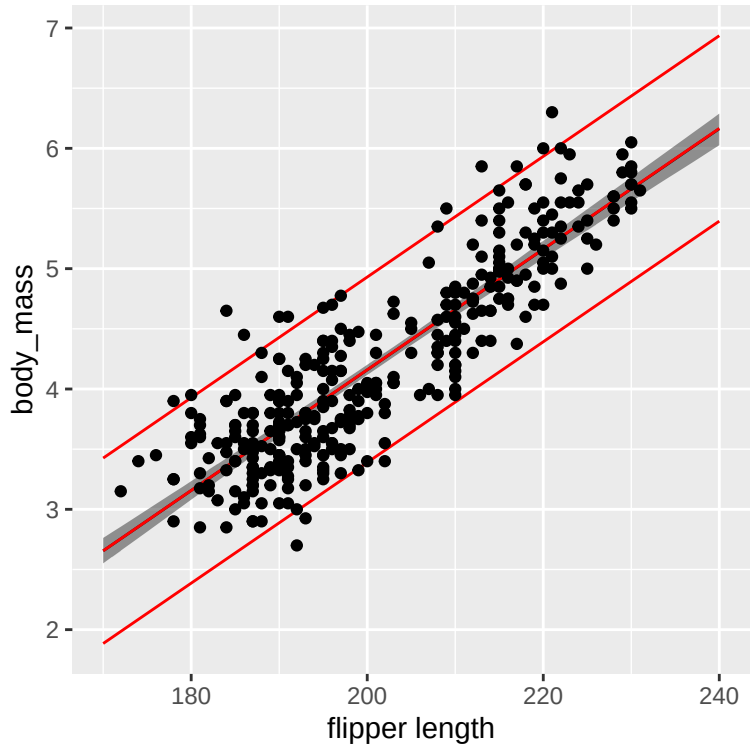


Figure 2: Data and 95% credible intervals

Task

Compute predictions for the mean body mass η when the flipper length is $x_0 = 193$ mm
What is the predicted value for η? And what is the uncertainty?
Take hint

You can create a new data frame containing the new observation x_0 and then use the `predict` function.
Click here to see the solution

```
new_data = data.frame(flipper_len = 193)
pred = predict(fit1, new_data, ~ beta_0 + beta_1,
               n.samples = 1000)

pred
```

```
  flipper_len     mean         sd  q0.025    q0.5   q0.975  median
1         193 3.806817 0.02544896 3.75613 3.80662 3.857113 3.80662
  mean.mc_std_err sd.mc_std_err
1    0.0008398828  0.0005552342
```

You can see the predicted mean and sd by examining the produced `pred` object. In this case the mean is ca 4 kg with sd ca 0.03. This gives a 95% CI ca [3.76, 3.86].

Task

On the previous task you computed a credible interval for the expected mean η when the flipper length is $x_0 = 193$ mm. Now produce a 95% *prediction* interval for a new observation y_0 (i.e., prediction interval for the body mass of a new penguin's whose flipper length is $x_0 = 193$ mm) by adding the uncertainty that comes from the likelihood with precision $\tau_y = 1/\sigma_y^2$
Take hint
You can use the `bru_names(fit1)` function to check the names for the different model components.
Click here to see the solution

```
pred2 = predict(fit1, new_data,
                formula = ~ {
                  mu = beta_0 + beta_1
                  sigma = sqrt(1/Precision_for_the_Gaussian_observations)
                  list(q1 = qnorm(0.025, mean = mu, sd = sigma),
                       q2 =  qnorm(0.975, mean = mu, sd = sigma))},
                n.samples = 1000)
# Notice that now the interval we obtain is much bigger!
round(c(pred2$q1$mean, pred2$q2$mean),2)
```

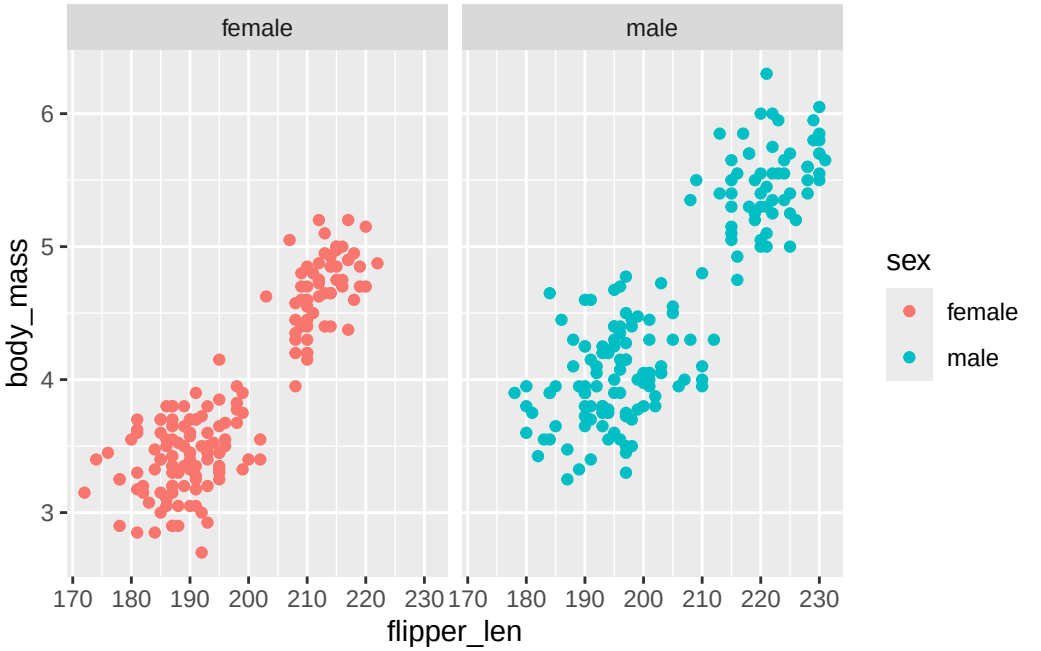
```
[1] 3.04 4.58
```

3 Linear model with discrete variables and interactions

Now we want to check if there is any difference, both in mean body_mass and in mean increase of the body_mass with flipper length, between males and females

Let's first look at an exploratory plot

```
penguins %>%
  ggplot() + geom_point(aes(flipper_len, body_mass, color= sex)) +
  facet_wrap(.~sex)
```



In the previous exercise we use `model = "linear"` to define each model component. Alternatively, `inlabru` features a `model = 'fixed'` component type that allows users to specify linear fixed effects using a formula as input. The basic component input is a model matrix. One can supply a formula specification, which is then used to generate a model matrix automatically. For example, we could define a model where the penguins body mass depends on the sex, which is a categorical variable:

$$y_i \sim \mathcal{N}(\mu_i, \sigma_y),$$
$$\mu_i = \eta_i = \beta_0 + \beta_{0,\text{Male}}$$

Here, the intercept β_0 is the mean body mass of female penguins, which is the reference level of `sex`. The coefficient $\beta_{0,\text{Male}}$ is the difference in mean body mass between males and females, so the mean body mass of male penguins is given by $\beta_0 + \beta_{0,\text{Male}}$. We can fit this model as follows:

```
cmp = ~ -1 + effects( ~ sex , model = "fixed")

formula = body_mass ~ effects

lik = bru_obs(formula = formula,
              data = penguins
              )
fit2 = bru(cmp, lik)
```

Specifying components this way, generate a model matrix automatically via `MatrixModels::model.Matrix(formula, data = .data.)`. Also notice that we have explicitly removed the global intercept from the model components. This is because specifying `model = "fixed"` calls the `model.Matrix` function which automatically creates an `(Intercept)` column in the design matrix.



> **Note**
>
> Note that for implementation technical reasons, `model = "fixed"` will make the estimated parameters to appear in `summary.random` instead of the normal `summary.fixed` part of the `inlabru` output object. E.g.,

```
fit2$summary.random
```

```
$effects
           ID      mean         sd 0.025quant  0.5quant 0.975quant      mode
1 (Intercept) 3.8622625 0.05679553   3.750822 3.8622625  3.9737026 3.8622625
2     sexmale 0.6834199 0.07996156   0.526525 0.6834199  0.8403149 0.6834199
            kld
1 5.749272e-10
2 5.752043e-10
```

This syntax allows users to define individual fixed effects and their interactions concisely, drawing on functionality already provided by the `MatrixModels` package. For example, we could no instead allow the effect of flipper length's to depend on sex by having an interaction between `flipper_len` and 'sex:

$$
\begin{aligned}
y_i &\sim \mathcal{N}(\eta_i, \sigma_y^2) \\
\eta_i &= \beta_0 + \beta_{0,\text{Male}} + \beta_1 x_i + \beta_{1,\text{Male}} x_i
\end{aligned}
$$

Here x_i is the flipper length of the ith penguin.

- For females, the mean body mass follows a straight line with intercept β_0 and slope β_1.
- For males, the line has intercept $\beta_0 + \beta_{0,\text{Male}}$ and slope $\beta_1 + \beta_{1,\text{Male}}$.

So $\beta_{0,\text{Male}}$ and $\beta_{1,\text{Male}}$ measure how much the male intercept and slope differ from the female ones.

We fit this model using the `model = "fixed"` option:

```
cmp = ~ -1 + effects( ~ sex*flipper_len, model = "fixed")
formula = body_mass ~ .
lik = bru_obs(formula = formula,
              data = penguins
              )
fit3 = bru(cmp, lik)
```

> **Note**
>
> Note that specifying a full stop in the linear predictor `~ .` is equivalent to summing all the components defined in the model specification. Instead of listing each component by name, the dot acts as a shorthand for "*include every component*" so `~ .` expands to the sum of all the effects you defined in `cmp`.

We can get the results looking at the `summary.random` object as follows:

```
fit3$summary.random
```

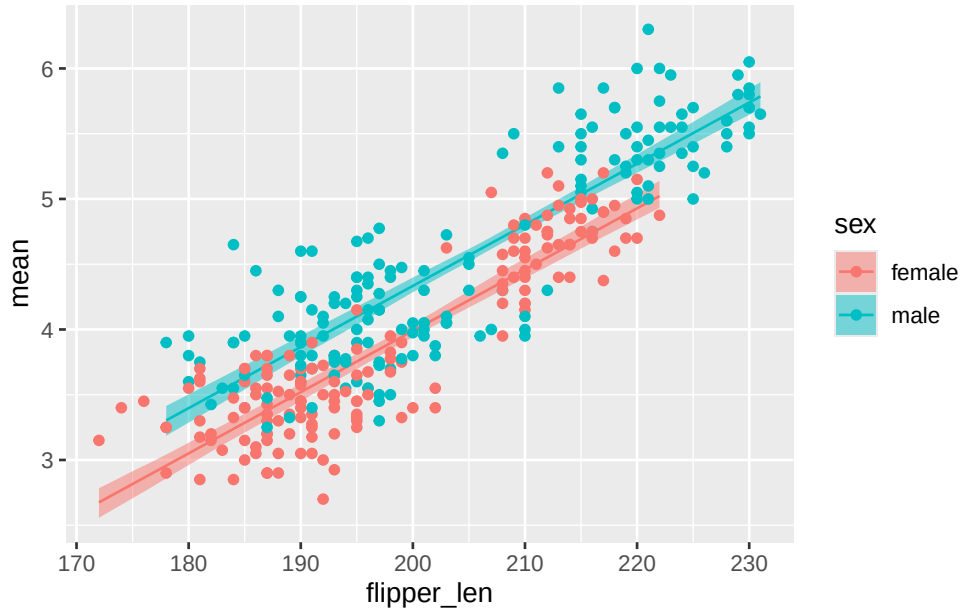
```
$effects
                   ID          mean           sd   0.025quant      0.5quant
1         (Intercept) -5.4428282993 0.439939195 -6.306039245 -5.4428316919
2             sexmale  0.4056078830 0.586824845 -0.745832499  0.4056114582
3         flipper_len  0.0471470109 0.002224650  0.042781896  0.0471470281
4 sexmale:flipper_len -0.0002882055 0.002921828 -0.006021178 -0.0002882235
   0.975quant          mode          kld
1 -4.57959800 -5.4428316871 5.830056e-10
2  1.55702787  0.4056114532 5.828004e-10
3  0.05151203  0.0471470280 5.829866e-10
4  0.00544487 -0.0002882235 5.827966e-10
```

Task

Use the `predict()` function to obtain the two regression lines for each sex. Would you conclude that there are differences between males and females?
Click here to see the solution

```
pred = predict(fit3, penguins, ~ effects)

pred %>% ggplot() + geom_line(aes(flipper_len, mean, group = sex, color = sex)) +
  geom_ribbon(aes(flipper_len, ymin = q0.025, ymax = q0.975,
                  group = sex, fill = sex), alpha = 0.5) +
  geom_point(aes(flipper_len, body_mass,color = sex))
```




```
# Differences on baseline body mass only, not in the slope.
```

Supplementary material

Another way to fit this model is to realize that a fixed effect can be seen as an iid effect with fixed precision, one of the `inlabru` ways to fit the model is:

```
#|
cmp = ~ -1 + sex_intercept(sex, model = "iid", initial = log(0.001), fixed = T) +
  sex_slope(sex, flipper_len,  model = "iid", fixed = T, initial = log(0.001))

lik = bru_obs(formula = body_mass ~ .,
              data = penguins)

fit3b = bru(cmp, lik)
```

Notice that we fix the precision of the `iid` effect to the same value as the precision of the linear effects which is 0.001.
The fitted values can be inspected as

```
fit3b$summary.random$sex_intercept
```

```
      ID      mean        sd 0.025quant  0.5quant 0.975quant      mode
1 female -5.442907 0.4399821  -6.306203 -5.442910  -4.579593 -
5.442910
2   male -5.036399 0.3884273  -5.798540 -5.036401  -4.274245 -
5.036401
           kld
1 5.835892e-10
2 5.837100e-10
```

```
fit3b$summary.random$sex_slope
```

```
      ID       mean          sd 0.025quant   0.5quant 0.975quant       mode
1 female 0.04714741 0.002224866 0.04278187 0.04714742 0.05151285 0.04714742
2   male 0.04685481 0.001894586 0.04313734 0.04685482 0.05057222 0.04685482
           kld
1 5.835741e-10
2 5.836388e-10
```

The results from the `fit3b` model are not the same we got from the `fit3` model. What is happening? It is just a matter of parametrization. You can go from one parametrization to the other by using the `predict()` function as we saw earlier.

4 Linear Mixed Model

When looking at the data we see that there are differences in body mass depending on the species.

```
penguins %>%
  ggplot() +
  geom_point(aes(flipper_len, body_mass, color = species)) +
  facet_wrap(.~sex)
```



To account for this we introduce `species` as a random effect in the model.

Task

Modify the code below to include an iid random effect for species:

```
cmp = ~ -1 + effects( ~ sex*flipper_len, model = "fixed") +
  species(species, ... )
formula = ...
lik = bru_obs(formula = formula,
              data = penguins
              )
fit4 = bru(cmp, lik)
```

Click here to see the solution

```
cmp = ~ -1 + effects( ~ sex*flipper_len, model = "fixed") +
  species(species, model = "iid")

formula = body_mass ~ .
lik = bru_obs(formula = formula,
              data = penguins
              )

fit4 = bru(cmp, lik)
```

We now look at the results for the fixed effects and compare with those from the previous model

```
fit4$summary.random$effects[,c(1,3,5)]
```

```
                  ID          sd     0.5quant
1         (Intercept) 0.727661182  0.351088128
```

```
2             sexmale 0.489338773 -0.351995483
3         flipper_len 0.003410089  0.017629134
4 sexmale:flipper_len 0.002449754  0.004398576
```

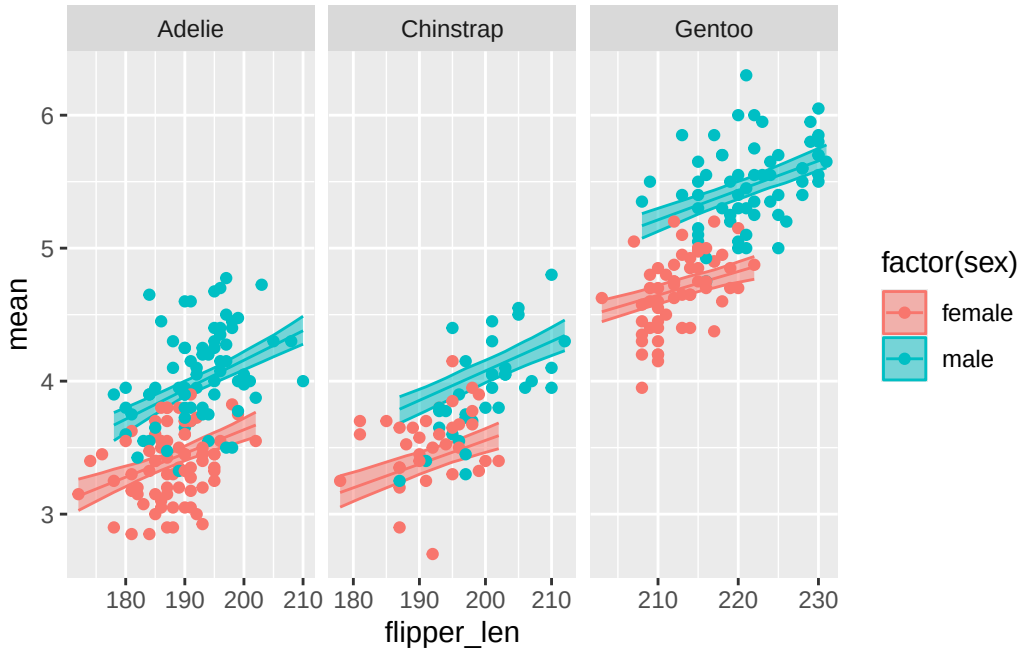
```
fit3$summary.random$effects[,c(1,3,5)]
```

```
                  ID          sd      0.5quant
1         (Intercept) 0.439939195 -5.4428316919
2             sexmale 0.586824845  0.4056114582
3         flipper_len 0.002224650  0.0471470281
4 sexmale:flipper_len 0.002921828 -0.0002882235
```

We can also visualize the fitted values using the `predict` function as we did before:

```
# Predict
pred = predict(fit4, penguins, formula = ~ effects + species)

pred %>%
  ggplot(aes(x=flipper_len,y=mean,color=factor(sex)))+
  geom_line()+
  geom_ribbon(aes(flipper_len,ymin = q0.025, ymax= q0.975,fill=factor(sex)), alpha = 0.5) +
  geom_point(data=penguins,aes(x=flipper_len,y=body_mass,colour=factor(sex)))+
  facet_wrap(~species,scales="free_x")
```



Task

We want to include a random effect, dependent on species, also on the slope. Modify the code below to include that random effect.

```
cmp = ~ -1 + effects( ~ sex*flipper_len, model = "fixed") +
  species1(species, ... ) +
  species2(species, ... )

formula = ...

lik = bru_obs(formula = formula,
              data = penguins
              )

fit5 = bru(cmp, lik)
```

Click here to see the solution

```
#|
cmp = ~ -1 + effects( ~ sex*flipper_len, model = "fixed") +
  species1(species, model = "iid") +
  species2(species, flipper_len, model = "iid")

formula = body_mass ~ .

lik = bru_obs(formula = formula,
              data = penguins
              )

fit5 = bru(cmp, lik)
```

5 Model comparison

As a by-product of the main computations `inlabru` can compute some scores that can be useful to compare models (we will talk more about this later in the course).

The easiest scores are the Deviance Information Criteria (DIC) and the Wakaike Information Criteria (WAIC).

These scores take into account goodness-of-fit and a penalty term that is based on the complexity of the model via the estimated effective number of parameters.

The lower the value of the score, the better the model is rated.

```
deltaIC(fit1, fit2, fit3, fit4, fit5, criterion = c("DIC","WAIC"))
```

```
Warning in deltaIC(fit1, fit2, fit3, fit4, fit5, criterion = c("DIC", "WAIC")):
WAIC values from INLA are not well-defined for point process models. Use with
caution, or better, not at all.TRUE
```

```
  Model      DIC  Delta.DIC     WAIC Delta.WAIC
1  fit5 137.3297   0.000000 137.0945   0.000000
2  fit4 138.6549   1.325213 138.4345   1.340023
3  fit3 263.9099 126.580239 263.6026 126.508134
```

```
4  fit1 327.5693 190.239634 327.5294 190.434938
5  fit2 739.3812 602.051580 738.8474 601.752971
```

6 Broom-style compatibility

From version `2.14.1.9005` onward, `inlabru` provides broom-style tidiers for `bru` model objects. If you already use the `broom` package, these functions return model summaries as tidy data frames that are easy to manipulate and plot. They are entirely optional for this course and are not needed to fit or work with models in `inlabru`.

To see how they work, let's revisit the simple linear regression model we fitted in @SLR.

```
cmp =  ~ -1 + beta_0(1) + beta_1(flipper_len, model = "linear")
eta = body_mass ~  beta_0 + beta_1
lik = bru_obs(formula = eta,
              data = penguins,
              family = "gaussian")
fit1 = bru(cmp, lik)
```

6 Posterior Summaries

We can pass `inlabru` objects to the `tidy()` function to produce posterior summaries of the hyperparameters and fixed effects in a `tibble` format:

```
tidy(fit1)
```

```
# A tibble: 2 x 5
  term   estimate std.error conf.low conf.high
  <chr>     <dbl>     <dbl>    <dbl>     <dbl>
1 beta_0  -5.87     0.310    -6.48     -5.26
2 beta_1   0.0502   0.00154   0.0471    0.0532
```

```
tidy(fit1, "hyperpar")
```

```
# A tibble: 1 x 5
  term                                    estimate std.error conf.low conf.high
  <chr>                                      <dbl>     <dbl>    <dbl>     <dbl>
1 Precision for the Gaussian observations     6.50     0.504     5.55      7.53
```

6 Predictions

An easy way to obtain fitted or predicted values using `augment()` function, which extends the original data with posterior fitted values, or adds posterior summaries of the linear predictor to a prediction grid.

For example, we can compute the expected mean $\mathbb{E}(\widehat{\mu_i}) = \widehat{\beta_0} + \widehat{\beta_1}$ flipper length using our existing data:

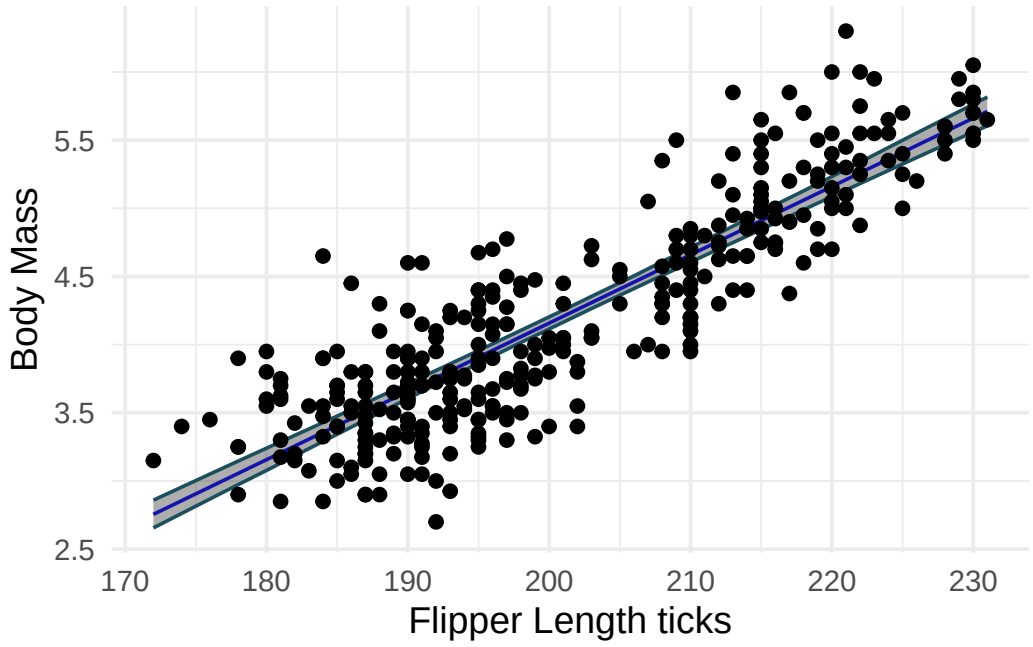
```
fitted_values <- augment(
  fit1,
  data = penguins,
  pred_formula = ~ beta_0 + beta_1,
  n_samples = 500L,
  seed = 1L
)

head(fitted_values)
```

```
# A tibble: 6 x 12
  species island    bill_len bill_dep flipper_len body_mass sex     year .fitted
  <fct>   <fct>        <dbl>    <dbl>       <int>     <dbl> <fct>  <int>   <dbl>
1 Adelie  Torgersen     39.1     18.7         181      3.75 male    2007    3.21
2 Adelie  Torgersen     39.5     17.4         186      3.8  female  2007    3.46
3 Adelie  Torgersen     40.3     18           195      3.25 female  2007    3.91
4 Adelie  Torgersen     36.7     19.3         193      3.45 female  2007    3.81
5 Adelie  Torgersen     39.3     20.6         190      3.65 male    2007    3.66
6 Adelie  Torgersen     38.9     17.8         181      3.62 female  2007    3.21
# i 3 more variables: .fitted_low <dbl>, .fitted_high <dbl>, .fitted_sd <dbl>
```

Then we can plot the expected mean as follows:

```
#|
ggplot(fitted_values, aes(x = flipper_len , y = .fitted)) +
  geom_line(col="blue")+
  geom_ribbon(aes(ymin = .fitted_low, ymax = .fitted_high),
                  alpha = 0.4, colour = "#1B4F5E") +
  geom_point(aes(x = flipper_len , y = body_mass )) +
  labs(x = "Flipper Length ticks", y = "Body Mass") +
  theme_minimal(base_size = 14)
```



The `augment()` function in `inlabru` calls `predict()` which in turn calls `generate()` to draw posterior samples.

6 Goodness-of-Fit

We can also use the `glance()` function to obtain our model's goodness-of-fit metrics in a tibble format (provided we set `bru_options_set(control.compute = list(dic = TRUE, waic = TRUE))`).

```
glance(fit1)
```

```
# A tibble: 1 x 5
    dic  waic marginal_loglik  nobs elapsed
  <dbl> <dbl>           <dbl> <int>   <dbl>
1  328.  328.           -187.   333    2.79
```